2024-2025

DECODE

DEtection of **CO**gnitive **D**istortions in **E**xpressions

BY :

SUSANNA MAZZOCCHI (535996)

Text Mining and Natural Language Processing--
Bachelor of Science in Artificial Intelligence

1. Introduction and Objectives

The purpose of DECODE is to detect **cognitive distortions** in textual evidence from dialogues of psychological counselling. Cognitive distortions are **irrational or biased ways of thinking that usually lead to psychological distress**, i.e., anxiety or depression.

For this purpose, I utilize the Therapist Q&A dataset on Kaggle, that comprises psychologically tagged examples of questions and answer used in counselling. Every example from the dataset is tagged with the absence or presence of some cognitive distortions.

The main objective of the project is to solve a **binary classification issue**, where each text sample will be classified as having or lacking cognitive distortions.

---

2. Data Analysis

The original dataset includes the following columns:

- **Id_Number**: Unique identifier for each sample.
- **Patient Question**: The text of the patient's question.
- **Distorted part**: The specific part of the text containing distortion (if applicable).
- **Dominant Distortion**: The main type of cognitive distortion detected (if any).
- **Secondary Distortion (Optional)**: An additional distortion type, if present.

Missing Values:

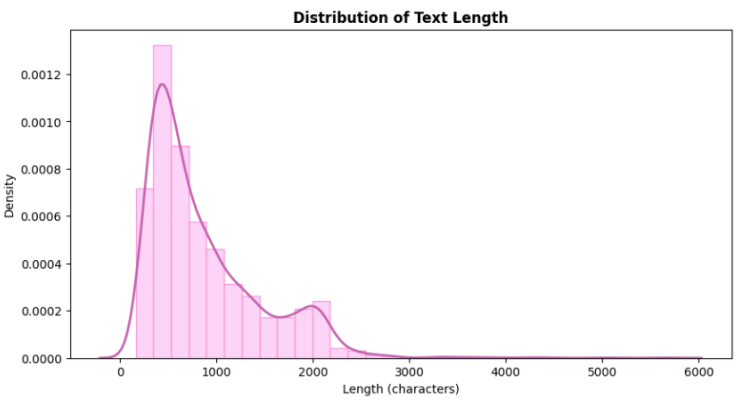
The columns ***Distorted part*** and ***Secondary Distortion (Optional)*** contained a large number of missing values (933 and 2,114 respectively) and were therefore **removed** from the dataset, as they were not essential for the binary classification task. The remaining columns — *Id_Number*, *Patient Question*, and *Dominant Distortion* — did not contain any missing values. However, the ***Id_Number*** column, being a simple identifier, was also dropped during preprocessing since it did not provide useful information for model training. As a result, only the *Patient Question* and *Dominant Distortion* columns were retained for further analysis.

Statistics:

Total number of samples: 2,530

For the binary classification task, the target variable **label** was constructed as follows:

- **0**: "No Distortion"
- **1**: Any type of distortion (all other labels)



Average text length (in number of characters): 869.8

Minimum text length: 161

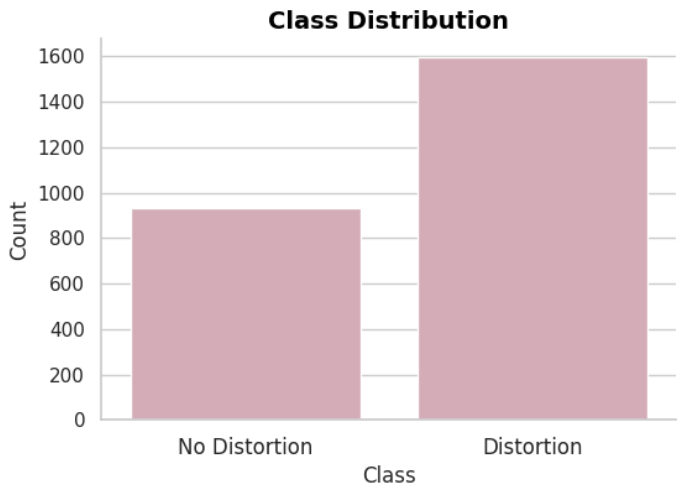
Maximum text length: 5,657

Median text length: 654

The histogram shows the distribution of text lengths in the dataset. Most texts are relatively **short**, but some **longer** entries are present, as expected for open-ended patient questions.

Class distribution:

Class	Count	Percentage
Distortion (1)	1,597	63.12%
No Distortion (0)	933	36.88%



The dataset is imbalanced, with 1,597 samples (63.12%) labelled as **"Distortion"** and 933 samples (36.88%) labelled as **"No Distortion"**. This imbalance was expected, considering the nature of the dataset, which consists of real patient questions submitted to an online therapy platform.

Since the purpose of these questions is most likely to express negative thoughts, express concerns, doubts, it's natural to have a higher percentage of the texts somehow engage in cognitive distortion.

The "No Distortion" label, on the other hand, is reserved for questions in which no apparent sign of distorted thinking occurs, something that naturally happens less frequently in this context.

Word Cloud:

To explore the most frequent words in the dataset, a **word cloud** was generated after applying the first step of text preprocessing which includes tokenization and lemmatization. This visualization demonstrates the words that most commonly appear in the corpus after cleaning and normalization. The following is the word cloud of the pre-processed text data:

3. Methodology

3.1 Text Preprocessing:

In order to prepare the dataset for modelling, a number of text preprocessing operations were carried out to **clean** and **standardize** the input. To minimize case variation, all the text was first **converted to lower case**. Then, URLs and user mentions were **removed** using regular expressions to eliminate irrelevant tokens.

Numbers, punctuation characters, including standard symbols and typographic quotation marks and dashes, were removed by replacing them with whitespace. Multiple consecutive spaces were collapsed into a single space, and leading/trailing spaces were stripped. Then, stop words were eliminated through the assistance of NLTK English stop words list, which removed common words containing little semantic content.

After cleaning, each text was **tokenized** into words, and **lemmatization** was applied to reduce words to their base or dictionary form. Lemmatization improves token consistency by grouping different inflected forms of a word under a single lemma, which improves feature quality for the models.

Original Sentence	Cleaned Text	Lemmatized Tokens
Hello, I have a beautiful, smart, outgoing...	hello beautiful smart outgoing amazing five year old...	[hello, beautiful, smart, outgoing, amazing, five, year, old, ...]

3.2 Text Representation:

After the text data had been pre-processed, the next step was to convert the cleaned textual data into numerical feature representations suitable for machine learning models. In this project, I experimented with three different techniques for text representation: **Positive Pointwise Mutual Information (PPMI)**, **Term Frequency-Inverse Document Frequency (TF-IDF)**, and **Word2Vec**.

1. PPMI Matrix

I created a co-occurrence matrix using word pairs within a context window. The PPMI transformation was employed to convert the matrix to the strength of association between word pairs while discarding negative values that indicate less meaningful co-occurrences.

In order to reduce the high dimensionality that is characteristic of the PPMI matrix, I performed **Singular Value Decomposition (SVD)**, projecting the data into a lower-dimensional representation while keeping most of the important information. This reduced representation was then used as a feature set that would be used in classification subsequently.

2. TF-IDF Representation

I also extracted **TF-IDF (Term Frequency-Inverse Document Frequency)** features from the cleaned text. TF-IDF encodes the importance of a word in a document relative to its frequency across the entire corpus, weighting useful terms more and common words less.

Similar to PPMI, the high dimensionality of the TF-IDF matrix was addressed by applying **SVD**, resulting in a low-dimensional feature set suitable for classification.

3. Word2Vec Embeddings

For Word2Vec, I trained a model on the lemmatized tokens extracted during the preprocessing step. The **Skip-Gram architecture** was used to predict surrounding words within a specified window size, capturing semantic relationships between words.

Each document was then represented by computing the **mean of the Word2Vec embeddings** for all words in the document. This approach provides a dense and informative vector representation of the entire text while preserving semantic similarities.

3.3 Classification Task:

Initially, **Logistic Regression** was applied to each of the three representations. This algorithm was used because it is simple, easy to interpret, and can handle binary classification problems. The implementation was made using scikit-learn's `LogisticRegression` class with default parameters, except for the random seed fixed at 42 to guarantee reproducibility. The three feature sets (PPMI, TF-IDF, and Word2Vec) were each run through the classifier, and the models were compared on the same held-out test set with accuracy, precision, recall, and F1-score.

The **SVM** model was implemented using scikit-learn's `SVC` class. A linear kernel and a radial basis function (RBF) kernel were both applied to examine the model's capability of identifying either linear or more intricate, non-linear relationships within the dataset.

The main hyperparameters included the penalty parameter C, which was set to 1.0, and gamma set to 'scale' for the RBF kernel, whereas the linear kernel was applied with default parameters.

In addition to SVM, a **Random Forest classifier** was trained using scikit-learn's `RandomForestClassifier`. The number of trees was set to 100, and the random seed fixed at 42, with other parameters left at their defaults. This ensemble method was chosen for its resilience to noise and for being capable of handling potentially complicated patterns in the TF-IDF features.

For all the models, a stratified train-test split of 80% training and 20% testing was used to maintain class balance. Moreover, five-fold cross-validation was applied during training to increase model reliability and prevent overfitting. Evaluation metrics used included accuracy, precision, recall, and F1-score for overall model performance assessment.

This approach made it possible to determine the best pair of feature representation and classifier that formed the basis for the final model selection in the study.

Bert:

To explore the transformer-based models, I decided to try with BERT, DistilBERT, and RoBERTa architectures using the `transformers` library. After preliminary tests, **RoBERTa-base** was selected due to its superior performance on our dataset.

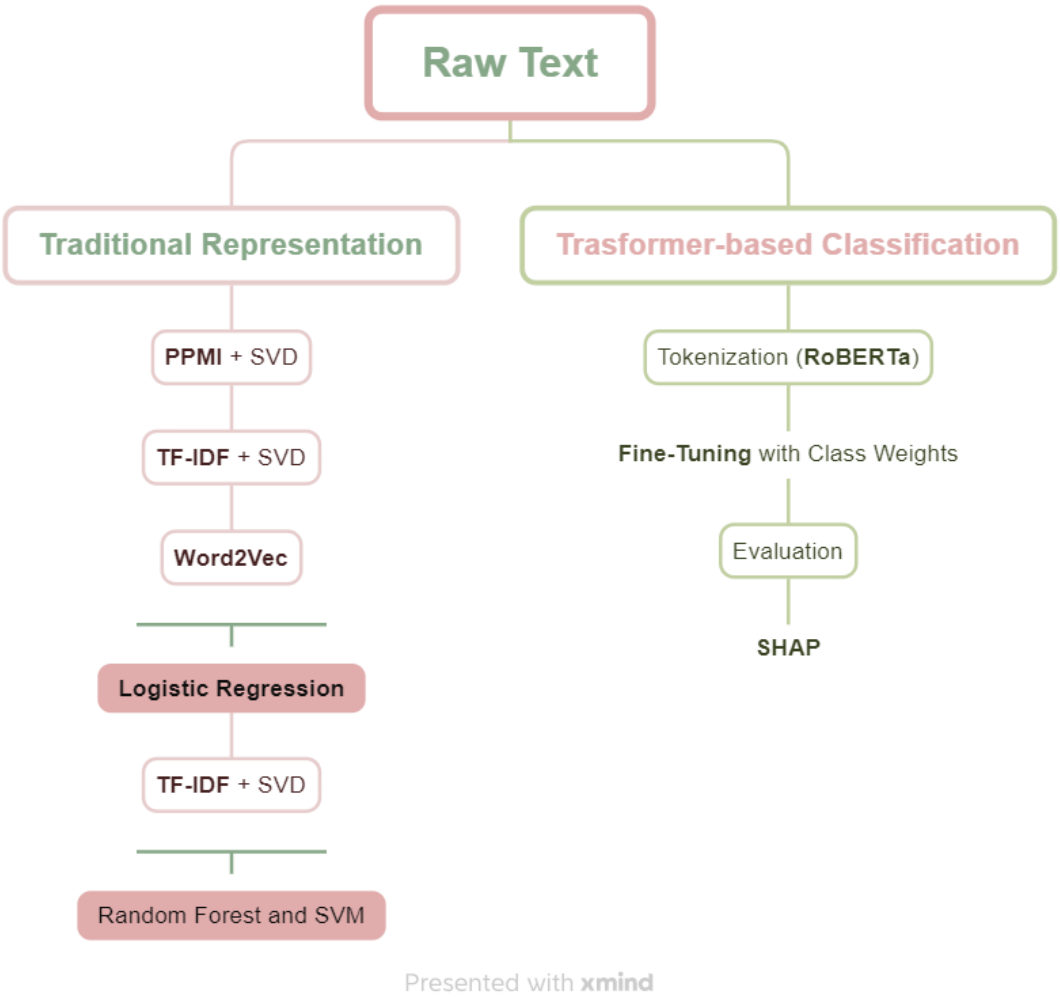
```
model_name = "roberta-base"

tokenizer = AutoTokenizer.from_pretrained(model_name)


model = AutoModelForSequenceClassification.from_pretrained(

    model_name,

    num_labels=2,

    id2label={0: "No Distortion", 1: "Distortion"},

    label2id={"No Distortion": 0, "Distortion": 1}

)
```

The dataset was split into training and validation sets (80/20) while preserving class distribution via stratification. Texts were tokenized using the corresponding RoBERTa tokenizer with truncation and padding up to a maximum length of 512 tokens.

A custom Trainer class was implemented to integrate **class weighting into the cross-entropy loss**, addressing the dataset's class imbalance. The model was trained with AdamW optimizer and learning rate of 2e-5 for 5 epochs. Batch size (16) and weight decay (0.01) were hyperparameters which were tuned based on validation performance.

For evaluation, standard metrics such as **accuracy, precision, recall, and F1-score** were computed. In addition to this, custom text samples, both from the dataset and created manually, were used to explore model behaviour.

Lastly SHAP (Shapley Additive exPlanations) was employed for explaining model predictions to understand the contribution of each token towards decision-making for classification.



"Overall workflow combining traditional feature-based approaches (TF-IDF, PPMI, Word2Vec) and transformer-based fine-tuning (RoBERTa), including evaluation and explainability steps."

4. Results and Analysis

4.1 Overview of Experiments

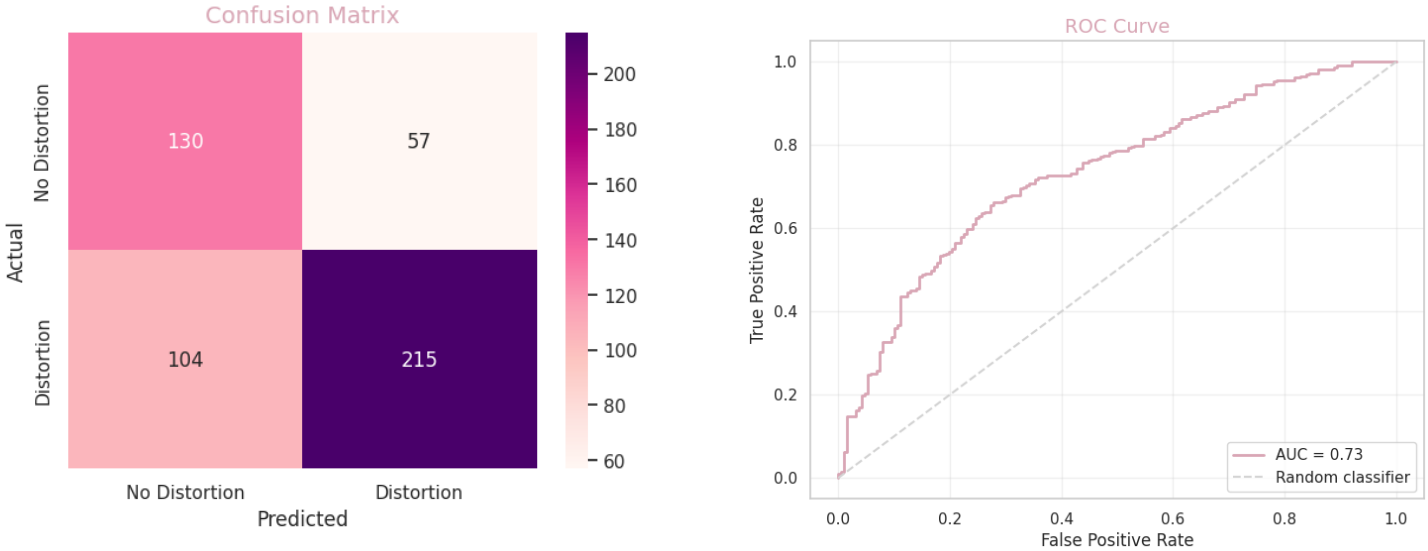
In this section, I present the performance of various models and text representation approaches evaluated on the cognitive distortion detection task. The results are analysed both quantitatively, through standard evaluation metrics, and qualitatively, by manually inspecting selected model outputs. Interpretability methods are also applied to gain insights into model behaviour.

4.2 Results of Classical Machine Learning Approaches

PPMI + Logistic Regression

Metric	Class 0	Class 1	Macro Avg	Weighted Avg
Precision	0.56	0.79	0.67	0.70
Recall	0.70	0.67	0.68	0.68
F1-score	0.62	0.73	0.67	0.69
Accuracy	–	–	–	**0.68**

Performance metrics for PPMI + Logistic Regression



Confusion Matrix Heatmap and ROC curve for PPMI + Logistic Regression

Analysis:

The PPMI-based representation combined with Logistic Regression produced an overall accuracy of **68%**, with a relatively balanced performance across both classes.

- **Class 0 (No Distortion):** Achieved a precision of **0.56** and a recall of **0.70**, indicating that while the model captures most of the true negatives, it does so at the expense of some false positives.

- **Class 1 (Distortion):** Showed stronger performance with a precision of **0.79** and a recall of **0.67**, meaning the model is more confident in predicting distortions but misses some true positives.

The confusion matrix reveals that:

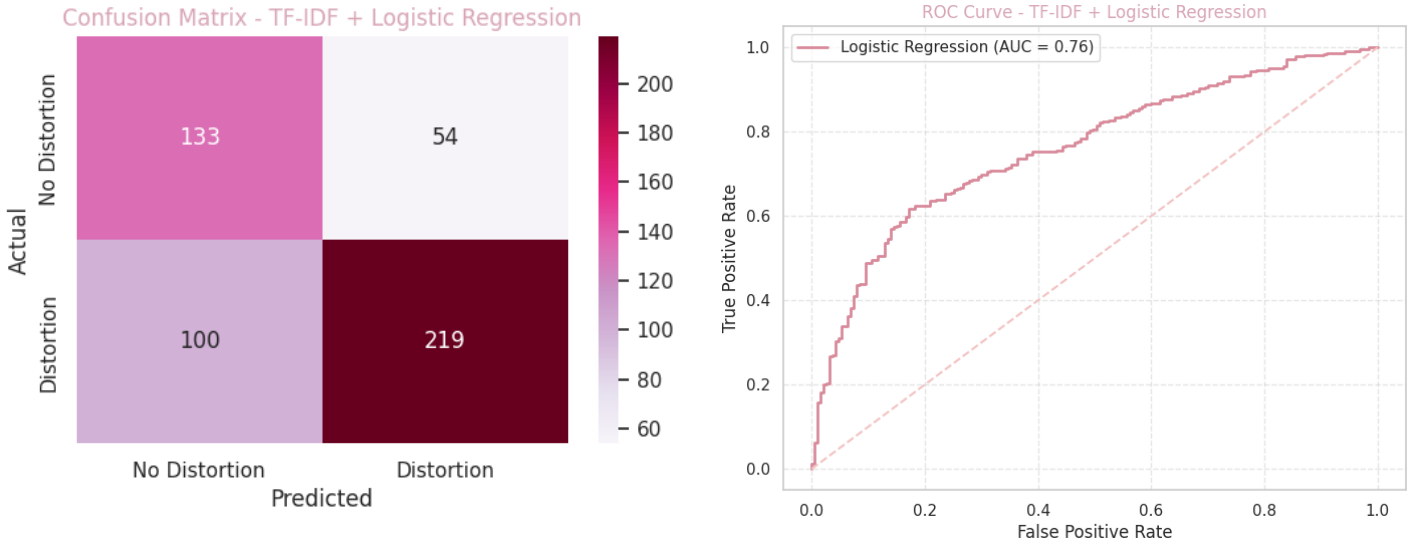
- **57 samples** of class 0 were misclassified as distortions.
- **104 samples** of class 1 were misclassified as non-distortions.

The ROC curve indicates an AUC of **0.73**, suggesting a modest but useful discriminative capability.

TF-IDF + Logistic Regression

Metric	Class 0	Class 1	Macro Avg	Weighted Avg
Precision	0.57	0.80	0.69	0.72
Recall	0.71	0.69	0.70	0.70
F1-score	0.63	0.74	0.69	0.70
Accuracy	–	–	–	**0.70**

Performance metrics for TF-IDF + Logistic Regression



Confusion Matrix Heatmap and ROC curve for TF-IDF + Logistic Regression

Analysis:

The TF-IDF-based representation combined with Logistic Regression achieved an overall accuracy of **69.6%**, slightly higher than the PPMI approach.

- **Class 0 (No Distortion):** Precision is **0.57**, recall is **0.71** — the model is relatively better at capturing true negatives but produces a fair number of false positives.
- **Class 1 (Distortion):** Shows higher precision (**0.80**) and a recall of **0.69**, confirming that the model is confident in identifying distortions but still misses some.

The confusion matrix indicates:

- **54 samples** of class 0 misclassified as distortions.
- **100 samples** of class 1 misclassified as non-distortions.

The ROC curve shows an AUC of **0.76**, suggesting that the model has a reasonable ability to distinguish between the two classes, slightly outperforming the PPMI model.

Word2Vec + Logistic Regression

Metric	Class 0	Class 1	Macro Avg	Weighted Avg
Precision	0.54	0.79	0.67	0.70
Recall	0.70	0.66	0.68	0.67
F1-score	0.61	0.72	0.66	0.68
Accuracy	–	–	–	**0.67**

Performance metrics for Word2Vec + Logistic Regression

Analysis:

The Word2Vec-based representation with Logistic Regression achieved an overall accuracy of **67.2%**, which is slightly lower than the TF-IDF approach.

- **Class 0 (No Distortion):** Precision is **0.54**, recall is **0.70**, indicating that the model tends to correctly detect most actual non-distortions but misclassifies some distortions as non-distortions.
- **Class 1 (Distortion):** Higher precision (**0.79**) but slightly reduced recall (**0.66**) compared to TF-IDF, meaning the model is precise in its positive predictions but misses some true positive distortions.

The confusion matrix shows:

- **57 samples** of class 0 misclassified as distortions.
- **109 samples** of class 1 misclassified as non-distortions.

In general, this approach performs reasonably well but does not outperform TF-IDF in terms of overall accuracy or F1-score.

Among the traditional text representation methods, **TF-IDF combined with Logistic Regression** demonstrated the best performance in terms of simplicity, stability, and balanced classification, achieving an accuracy of **69.6%** and an AUC of **0.76**. This approach outperformed both **PPMI + Logistic Regression** (68.2% accuracy, AUC 0.73) and **Word2Vec + Logistic Regression** (67.2% accuracy, AUC not computed), confirming the effectiveness of TF-IDF features for this task.

To further explore classifier choice, various models were tested using TF-IDF features:

- **Random Forest (default settings)** reached **69.4% accuracy**, with a good performance on the positive class (f1-score 0.78) but weaker results for the negative class (f1-score 0.52). The model tended to favor the majority class.

- **Random Forest (optimized via RandomizedSearchCV)** slightly improved performance to **71.1% accuracy**, offering more balanced results (f1-scores: 0.59 for class 0, 0.78 for class 1). Hyperparameter tuning helped mitigate overfitting and improved minority class recognition.
- **Support Vector Machine (optimized via GridSearchCV)** achieved **69.6% accuracy**, with an especially strong recall on the positive class (0.83) but lower recall on the negative class (0.46), indicating some difficulty in distinguishing minority samples.

Despite these variations, **Logistic Regression with TF-IDF remained the most balanced and interpretable choice**, showing consistent results across both classes without strong bias or overfitting issues. The tree-based models and SVM provided slightly higher scores for the positive class but struggled to generalize well across both categories, which is crucial in tasks involving cognitive distortion detection where false positives and false negatives are equally important to control.

4.3 Results of Transformer-based Approaches

Model	Accuracy	Precision	Recall	F1 Score
DistilBERT	69.0%	81.2%	74.3%	77.6%
BERT-base-uncased	70.3%	77.2%	75.2%	77.2%
RoBERTa-base	**77.1%**	**82.0%**	**81.5%**	**81.7%**

Final results achieved by each model at their best training epoch

Analysis:

The **RoBERTa-base model outperformed the others across all key metrics**:

- **Highest Accuracy (77.1%)**
- **Best F1 Score (81.7%)**: Demonstrates a strong balance between precision and recall, which is crucial for this task to ensure both correct detection and low false-negative rates.
- **Significant Recall improvement (81.5%)**: This is especially important in the therapeutic context, where missing instances of cognitive distortions could be detrimental.
- **High Precision (82.0%)**: Suggests that the model is not producing excessive false positives.

Final Model Selection

From this overall evaluation, **RoBERTa-base was chosen as the final model** for this project. Its higher performance in F1 Score and Recall especially makes it the best option for the sensitive nature of cognitive distortion identification task.

Further improvements could still be explored, such as implementing class weighting, applying data augmentation techniques (e.g., synonym replacement or back translation), and fine-tuning the learning rate or using early stopping to prevent overfitting. However, even without these additional optimizations, RoBERTa has already demonstrated a substantial and reliable advantage over the other models tested.

RoBERTa-base (Selected Model)

The fine-tuned RoBERTa model achieved the following performance on the validation set after 5 epochs:

Metric	Value
Accuracy	0.7707
Precision	0.8201
Recall	0.8150
F1-score	0.8013
Validation Loss	0.8176

Throughout the training process, the model demonstrated steady improvement in precision and recall, suggesting a good balance between sensitivity and specificity. The final F1-score of 0.80 reflects robust performance on the cognitive distortion detection task.

Qualitative Error Analysis

To better understand the model's behaviour, several manually selected examples were tested:

Text Sample	Ground Truth	Model Prediction	Confidence	Comment
I always fail at everything	Distortion	Distortion	77.81%	-
The weather is nice today	No Distortion	No Distortion	88.55%	-
I started self harming when I was 12...	No Distortion	No Distortion	64.33%	Correct according to label, but semantically complex; self-harm implies psychological distress but not necessarily distorted cognition per dataset definition.
Could my boyfriend be bipolar?...	Distortion	Distortion	78.78%	-

Discussion:
One of the most interesting cases is the third example concerning self-harm. While one can intuitively sense that this sentence suggests internalized psychological issues, the dataset labels it as "No Distortion," possibly because the statement is a factual description of behaviour rather than a distorted thought pattern. RoBERTa correctly predicted this according to the dataset, indicating that it has effectively learned the dataset's labelling criteria—even when those criteria might not align perfectly with human intuition.

4.4 Explainability with SHAP

To better understand the predictions, I applied **SHAP (SHapley Additive exPlanations)**, a popular explainable AI (XAI) framework.

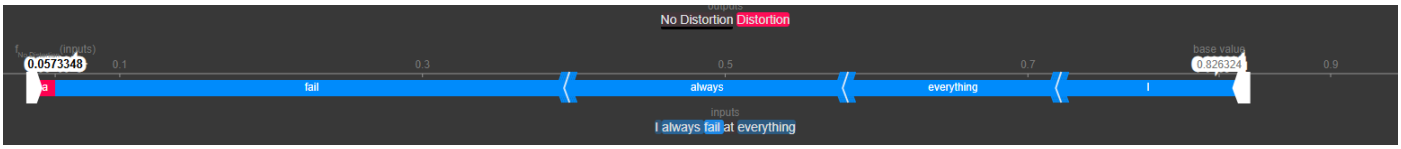
SHAP is a method based on cooperative game theory that attributes to each input feature a contribution to the final model prediction. In the context of NLP, it allows us to visualize how individual words in a sentence impact the model's output, either pushing it toward or away from a certain class. *https://shap.readthedocs.io/en/latest/*

I used SHAP to interpret the behaviour of our model, validate that the model is not relying on irrelevant or spurious patterns, understand better which phrases are most indicative of cognitive distortions according to the model.

Results

Below is a SHAP explanation for the sentence:

"I always fail at everything."



The words **"always"**, **"fail"**, and **"everything"** are highlighted as strong contributors toward the **"Distortion"** class. This is consistent with cognitive psychology, where such absolute and generalized statements are typical markers of cognitive distortions.

SHAP therefore confirms that the model is learning semantically relevant patterns and not relying on noise. This enhances our confidence in the interpretability and trustworthiness of the classifier.

5. Conclusion

5.1 Summary

The goal of this project was to detect cognitive distortions in user-generated text using both traditional machine learning approaches and modern transformer-based models. The experiment results highlighted numerous significant findings:

Among traditional approaches, **Logistic Regression when paired with TF-IDF representation** proved to be the most reliable and balanced method. While tree-based models like Random Forest and SVM achieved slightly higher performance on the positive (majority) class, they struggled to maintain generalization across both classes. In contrast, Logistic Regression provided stable and interpretable results, crucial for a task where both false positives and false negatives carry significant implications.

The transformer-based approach, **RoBERTa fine-tuned on our dataset**, achieved the highest overall performance in terms of accuracy and F1-score. However, this came at the cost of increased computational demands and reduced interpretability compared to simpler models. The model showed strong generalization and was able to handle complex text structures that traditional methods might overlook.

A close examination of BERT's predictions revealed that the model could correctly classify complex examples, although some borderline cases (such as discussions about self-harm) highlighted the complexity of the task and the potential ambiguity in human labelling itself.

5.2 Challenges Encountered

Throughout the development of this project, several challenges emerged:

- **Class Imbalance:**

The dataset was highly imbalanced, with a strong prevalence of the "Distortion" class over "No Distortion". This issue risked biasing the classifier toward the majority class, reducing the detection sensitivity for the minority class.

Solution: To help with this, class weighting was integrated into the loss function (in both Logistic Regression and the custom BERT Trainer), and stratified splits were employed to preserve class proportions during training and validation.

- **Computational Limitations with BERT Fine-Tuning:**

Fine-tuning transformer models like RoBERTa on Google Colab introduced significant constraints. On the free CPU runtime, a single epoch could take up to one hour, making iterative experimentation impractical. The GPU (Tesla T4) was only available for one hour per day under the free tier, which severely limited the ability to perform multiple runs or hyperparameter searches. This bottleneck made model optimization and debugging more time-consuming than expected. When GPU resources were available, training time was reduced to about 10 minutes per epoch.

- **Limited Dataset Size:**

Although not a critical issue, the dataset size was modest for training large models like RoBERTa. This could limit generalization capability, especially in edge cases not well represented in the training data.

5.3 Possible Improvements and Future Work

To enhance the outcome of this project, the following improvements are suggested:

- **Larger and Better-Balanced Datasets:**

Getting or generating additional annotated data—especially for the minority class—would help models generalize better and mitigate the dangers of class imbalance.

- **Data Augmentation:**

Employing NLP-specific data augmentation techniques (e.g., synonym replacement, back-translation) could expand the dataset artificially and reduce overfitting.

- **Multi-class or Multi-label Expansion:**

Future work can involve distinguishing between different types of cognitive distortions rather than a simple binary classification, and the output of the model will then be more applicable in psychological applications.

6. AI Policies and References

6.1 AI Policies

During the course of this project, I occasionally used Google Colab's Gemini AI assistant to debug Python code. This assistant helped identify the causes of certain errors, such as incorrect arguments, missing code blocks, or incorrectly set parameters, and providing explanations and recommendations for their fix. For example, Gemini clarified exceptions related to the use of HuggingFace's pipeline() function and setting up the Trainer.

All AI suggestions were carefully reviewed, verified, and manually tested before being added to the final version of the code. Additionally, CHAT GPT was also used to refine visualization aesthetics and readability, e.g., choosing colour schemes and plot layouts, to render graphical outputs clearer and more effective. AI was not used for report writing or interpretation of results, only for code debugging assistance and aesthetic purpose.

6.2 References

I drew inspiration and materials from various sources, including:

- https://www.kaggle.com/code/pritishmishra/fine-tune-bert-for-text-classification?scriptVersionId=116951029
- https://www.kaggle.com/code/lalwaniabhishek/using-bert-for-binary-text-classification
- https://penscola.medium.com/building-a-sentiment-analysis-model-with-three-powerful-models-roberta-bert-and-distilbert-24165582f7a3
- https://pytorch.org/text/0.17.0/datasets.html#text-classification